

Project 0: The Horror Marathon Ranker

A three-day warm-up assignment in Java, Spring Boot, PostgreSQL, React, Docker and CI

Issued: Tuesday, September 29, 2026

Due: Thursday, October 1, 2026, end of day

Live demo: Friday, October 2, 2026 (the marathon) **Estimated effort:** 14 to 18 hours

1. Overview

On Friday you and your friends will hold a horror movie marathon. Each person secretly picks one movie that nobody in the group has seen. The films are watched in a random order, everyone rates each film on a short rubric, and at the end the application produces a ranking from favourite to worst. Your job is to build that application.

The real purpose of this assignment is preparation. Your next project, the dispatch planner, uses the same stack at a much larger scale. Treat this week as a rehearsal: every configuration problem you solve now is one you will not have to solve later, when the problems are harder.

2. Learning objectives

By the end of this project you should be able to:

- Scaffold a Spring Boot 3 application on Java 21 and structure it into controller, service and repository layers.
- Run PostgreSQL (with the PostGIS extension enabled) in Docker Compose and manage the schema with Flyway migrations.
- Design a REST API with request and response DTOs that deliberately hide data from the client.
- Build a small React front end that talks to that API.
- Package the whole system in containers and run automated builds and tests with GitHub Actions.
- Deploy the application so other people can use it from their phones.

3. Functional requirements

Your finished application must support the following. Items marked (*stretch*) are optional.

ID	Requirement
R1	A user can create a group and receive a short join code (for example, K7QX).
R2	Friends join the group with the code and a nickname. No passwords or accounts.
R3	Each member searches a movie database and submits exactly one pick. Picks are anonymous : no API response may reveal who picked which film.
R3a	If a film is not found in the search, the member can add it by manual entry (title and year, with an optional poster URL). Manual picks behave exactly like searched picks everywhere else in the app.
R4	Any member can anonymously flag a pick as “I’ve seen this.” A flagged pick is removed and its owner is asked to choose again.
R5	When everyone has picked, the group is started and the watch order is randomized once and saved . Refreshing the page must not reshuffle it.
R6	After each film, every member rates it from 1 to 10 in each rubric category (Section 4).
R7	A member’s ratings of their own pick are excluded from that film’s score. The server knows the picker; the UI never shows it.
R8	A results page ranks all films from highest to lowest score once rating is complete.
R9	(<i>stretch</i>) After results are shown, reveal who picked each movie.

4. Rating rubric

Ten categories are too many to fill in five times in one night. Use these five, each scored 1 to 10. Critical reception has been removed because you are rating your own experience, and jump scares have been folded into scariness.

Category	What the rater is judging
Scariness	Overall fear factor, including how well the jump scares land.
Atmosphere	Dread, tension and disturbing imagery.
Story	Quality and coherence of the plot, and how memorable the characters are.
Acting	Performances and believability.
Enjoyment	How much you enjoyed it overall and whether you would watch it again.

Scoring rule. For each rater, average their five category scores. A film’s final score is the mean of those averages across all eligible raters (everyone except the picker). Break ties with the higher average Enjoyment score.

5. Reference design

You may change this design, but you should be able to justify any change. The goal is a schema you can finish in an evening.

5.1 Database schema

Table	Columns	Notes
groups	id, name, join_code, status, created_at	status is LOBBY, WATCHING or DONE. join_code is unique.
members	id, group_id, nickname, token	token is a random secret stored in the browser. Unique (group_id, nickname).
picks	id, group_id, member_id, source, tmdb_id, title, release_year, poster_url, watch_order, vetoed	source is TMDB or MANUAL. tmdb_id is null for manual picks; enforce this with a CHECK. watch_order is null until the group starts. Unique (group_id, member_id) for non-vetoed picks.
ratings	id, pick_id, member_id, scariness, atmosphere, story, acting, enjoyment	Each score has CHECK (score BETWEEN 1 AND 10). Unique (pick_id, member_id).

5.2 REST endpoints

Method and path	Purpose
POST /api/groups	Create a group. Returns the join code and the creator's member token.
POST /api/groups/{code}/members	Join with a nickname. Returns a member token.
GET /api/groups/{code}	Group state: status, member nicknames, picks (without picker), watch order.
GET /api/movies/search?q=	Proxy search to the movie database so the API key stays on the server.
POST /api/groups/{code}/picks	Submit or replace the caller's pick: either a TMDB id or a manual title and year.
POST /api/picks/{id}/veto	Flag a pick as already seen.
POST /api/groups/{code}/start	Lock picks, shuffle, save watch_order, set status to WATCHING.
PUT /api/picks/{id}/rating	Submit or update the caller's ratings for a film.
GET /api/groups/{code}/results	Ranked films with final scores.

The caller identifies itself with an X-Member-Token header. The server looks up the member from the token. This is not real security, and it does not need to be for a movie night.

6. Day 1 — Tuesday evening: foundation and data layer

Target: about 4 hours. Goal: a running API backed by a real database.

Part A. Repository and environment (45 minutes)

- 1 Create a GitHub repository named something like `horror-ranker` with two folders: `backend/` and `frontend/`. Add a `.gitignore` for Java, Node and IDE files.
- 2 Write a `docker-compose.yml` at the root with one service, `db`, using the `postgis/postgis` image, a named volume, and port 5432 exposed. Put credentials in a `.env` file and add it to `.gitignore`.
- 3 Run `docker compose up -d` and connect with `psql` or a GUI client. Run `CREATE EXTENSION IF NOT EXISTS postgis;` and `SELECT postgis_version();` to prove it works. You will not use PostGIS this week; you are confirming the setup for the dispatch planner.

Part B. Spring Boot scaffold (45 minutes)

- 4 Generate the project at `start.spring.io`: Maven or Gradle, Java 21, Spring Boot 3.x. Add Spring Web, Spring Data JPA, PostgreSQL Driver, Flyway Migration, Validation, Spring Boot Actuator and Testcontainers.
- 5 Configure `application.yml` to read the database URL, user and password from environment variables. Set `spring.jpa.hibernate.ddl-auto: validate` so Hibernate checks your schema instead of creating it.
- 6 Start the app and open `/actuator/health`. It should report UP, including the database.

Part C. Schema and entities (1 hour)

- 7 Write `V1__init.sql` in `src/main/resources/db/migration` with the four tables from Section 5.1, including foreign keys, unique constraints and CHECK constraints.
- 8 Create JPA entities and Spring Data repositories for each table. Use Java records for DTOs; never return an entity directly from a controller.
- 9 Restart the app and confirm Flyway applied V1 (check the `flyway_schema_history` table).

Part D. Groups and members (1.5 hours)

- 10 Implement `POST /api/groups` and `POST /api/groups/{code}/members`. Generate join codes and tokens with `SecureRandom`, and avoid ambiguous characters such as O, 0, I and 1 in join codes.
- 11 Write a small component that reads `X-Member-Token`, loads the member, and rejects requests whose token does not belong to the group in the URL (HTTP 403).
- 12 Implement `GET /api/groups/{code}`. Add a `@RestControllerAdvice` that turns not-found and validation errors into clean JSON responses with the right status codes.

Checkpoint. Using `curl` or an HTTP client, create a group, join it as two different members, and fetch its state. Commit and push. If you are behind schedule, Part D may spill into Wednesday morning, but Parts A to C may not.

7. Day 2 — Wednesday: game logic and the front end

Target: about 7 hours. Goal: a complete marathon can be played on localhost.

Part E. Movie search (45 minutes)

- 13 Register for a free API key from The Movie Database (TMDB). Store it as an environment variable, never in the repository.
- 14 Implement GET /api/movies/search by calling TMDB's movie search from the server with Spring's RestClient. Return only id, title, year and poster path.

Part F. Picks, manual entry, veto and start (2 hours)

- 15 Implement pick submission. A member may replace their pick while the group is in LOBBY; after that, picks are locked.
- 16 Support manual entry. The request carries either a TMDB id or a title and year, never both; validate this in the service and return 400 otherwise. Trim the title, cap its length, and accept a poster URL only if it starts with https://. For TMDB picks, look the title and poster up on the server rather than trusting the client.
- 17 Block duplicates: compare a normalized title (lowercase, punctuation stripped) plus year across the group, so the same film cannot sneak in once through search and once by hand. Reply with a neutral "that film is already picked" that does not reveal who picked it.
- 18 Implement veto. A vetoed pick disappears from the group view, and its owner's client should show "your pick was vetoed, choose again." Only the owner learns this, so anonymity holds.
- 19 Implement start: verify every member has a live pick, shuffle with `Collections.shuffle(list, new SecureRandom())`, write `watch_order` in a single `@Transactional` method, and set status to WATCHING.

Part G. Ratings and results (1 hour)

- 20 Implement rating submission with Bean Validation (`@Min(1) @Max(10)`) on each field. Reject a member rating their own pick, but return a generic response so the UI cannot use it to discover who picked what.
- 21 Implement results using the scoring rule in Section 4. Computing this in a single SQL query with `GROUP BY` is good practice for the dispatch planner; computing it in Java is also acceptable.

Part H. Tests (45 minutes)

- 22 Write unit tests for the scoring logic, including the self-rating exclusion and the tie-break.
- 23 Write at least one integration test with Testcontainers that starts a PostGIS container, runs Flyway, creates a group and plays through start and rating.
- 24 Write one test asserting that no pick in the group-state JSON contains a member id or nickname. This is your anonymity guarantee.
- 25 Test manual entry: a valid manual pick is accepted, a request with both a TMDB id and a title is rejected, and a manual duplicate of an existing TMDB pick is blocked.

Part I. React front end (3 hours)

- 26 Create the front end with Vite's React template. Configure the dev server to proxy /api to the backend so you avoid CORS problems.
- 27 Build five screens: **Home** (create or join), **Lobby** (members, pick status, search box, and a "Can't find it? Add it manually" form that appears when search returns nothing), **Watch order** (the shuffled list with posters, and a plain placeholder card for manual picks without one), **Rate** (five 1-10 inputs per film) and **Results**.
- 28 Save the member token in localStorage so a refresh does not log anyone out. Poll the group state every few seconds so everyone sees updates; do not build WebSockets this week.
- 29 Design for a phone first. Your friends will use this from the couch.

Checkpoint. On localhost, open three browser windows as three members and play a full marathon from create to results. Commit and push.

8. Day 3 — Thursday: containers, CI and deployment

Target: about 5 hours. Goal: a public URL your friends can open on Friday.

Part J. Containerize (1.5 hours)

- 30 Write a multi-stage backend/Dockerfile: a JDK 21 image to build the jar, then a smaller JRE 21 image to run it.
- 31 Choose how to serve the front end. The simplest option is to build the React app and copy the output into Spring Boot's static/ folder, giving you one container and one URL. Document your choice.
- 32 Add the app to docker-compose.yml and confirm docker compose up --build starts everything from a clean checkout.

Part K. GitHub Actions (1 hour)

- 33 Create .github/workflows/ci.yml that runs on every push and pull request: set up Java 21, cache dependencies, run the backend tests (Testcontainers works on GitHub's Ubuntu runners), then install and build the front end.
- 34 Add a step that builds the Docker image, so a broken Dockerfile fails CI. Add the status badge to your README.

Part L. Deploy (1.5 hours)

- 35** Pick a host that runs Docker containers and offers managed PostgreSQL, such as Render, Railway or Fly.io. Check current pricing; free tiers change often. Confirm the managed database supports PostGIS if you plan to reuse the host for the dispatch planner.
- 36** Set the database credentials and TMDB key as environment variables on the host. Deploy and confirm `/actuator/health` reports UP.
- 37** (Optional) Add a deploy job to the workflow that runs only on pushes to main after tests pass.

Part M. Dress rehearsal and write-up (1 hour)

- 38** Ask one friend to join a test group from their phone on mobile data. Fix whatever breaks. Something will.
- 39** Delete the test data, or add a way to create a fresh group, so Friday starts clean.
- 40** Write a README: what the app does, how to run it locally in one command, architecture notes, and a short list of what you would do differently.

Checkpoint. The deployed URL works on a phone, CI is green, and the README explains how to run the project. You are done.

9. Evaluation

You will be graded by your friends on Friday and by yourself on Saturday.

Criterion	Weight	Standard for full marks
Works on the night	35%	Everyone can join, pick, rate and see results without you touching a terminal.
Fairness	20%	Picks stay anonymous, order is random and stable, and self-ratings are excluded. Tests prove it.
Code quality	20%	Clear layers, DTOs, migrations, validation and consistent error handling.
Automation	15%	One-command local startup, green CI, repeatable deployment.
Documentation	10%	A README a stranger could follow.

10. Advice from the instructor

- **Protect the deadline, not the feature list.** If Thursday afternoon arrives and deployment is not working, cut R4 or the stretch goal. A deployed app with four features beats a local app with nine.
- **Deploy early if you can.** Pushing a “hello world” container to your host on Wednesday night removes most of Thursday’s risk.
- **Commit at every checkpoint.** Small commits with clear messages are part of what makes a repository look professional.
- **Keep a notes file.** Record every error you hit and how you fixed it. That file is the most valuable thing you will carry into the dispatch planner.
- **Late policy:** The marathon starts Friday. There are no extensions.